
JAVA OPERATORS

```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
#selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier  
mirror_ob.select = 0  
bpy.context.selected_object  
data.objects[one.name].se  
print("please select exactly  
  
-- OPERATOR CLASSES -----  
  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
error X"  
  
context):  
context.active_object is not
```

JAVA OPERATORS

Operators are used to perform operations on variables and values.

Java divides the operators into the following groups:

- Arithmetic operators
 - Assignment operators
 - Comparison operators
 - Logical operators
 - Bitwise operators
-

ARITHMETIC OPERATORS

- **Arithmetic operators** are utilized in mathematical expressions similarly that they are utilized in mathematics.

ARITHMETIC OPERATORS

Operator	Description
+	adds two operands
-	subtract second operands from first
*	multiply two operand
/	divide numerator by denominator
%	remainder of division
++	Increment operator increases integer value by one
--	Decrement operator decreases integer value by one

EXAMPLE –USING ARITHMETIC OPERATORS

Operator	Example (int A=8, B=3)	Result
+	A+B	11
-	A-B	5
*	A*B	24
/	A/B	2
%	A%4	0

EXAMPLE –USING ARITHMETIC OPERATORS

- In the example below, we use the + **operator** to add together two values:

```
int x = 100 + 50;
```

- Although the + operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

```
int sum1 = 100 + 50; // 150 (100 + 50)
```

```
int sum2 = sum1 + 250; // 400 (150 + 250)
```

```
int sum3 = sum2 + sum2; // 800 (400 + 400)
```

JAVA ASSIGNMENT OPERATORS

- **Assignment operators** are used to assign values to variables.
- In the example below, we use the **assignment** operator (=) to assign the value **10** to a variable called **x**:

```
int x = 10;
```

- The **addition assignment** operator (+=) adds a value to a variable:

```
int x = 10;
```

```
x += 5;
```

A LIST OF ALL ASSIGNMENT OPERATORS:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

ASSIGNMENT OPERATORS: SAMPLE PROGRAM

```
14 public class JavaApplication1 {
15
16     public static void main(String[] args) {
17         //+= Assignment Operator
18         int value1 = 4;
19         int value2 = 8;
20         value2 += value1;
21         System.out.println("value1 is " +value1);
22         System.out.println("value2 is " +value2);
23     }
24
25 }
```

javaapplication1.JavaApplication1 > main >

Output X

JavaApplication1 (run) x JavaApplication1 (run) #2 x

run:
***** += Assignment Operator*****
value1 is 4
value2 is 12

JAVA COMPARISON/RELATIONAL OPERATORS

- **Comparison operators**
are used to compare two
values:

Operator	Description
==	Check if two operand are equal
!=	Check if two operand are not equal.
>	Check if operand on the left is greater than operand on the right
<	Check operand on the left is smaller than right operand
>=	check left operand is greater than or equal to right operand
<=	Check if operand on left is smaller than or equal to right operand

EXAMPLE- COMPARISON/RELATIONAL OPERATORS

- Assume variable A holds 10 and variable B holds 20, then:

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.

EXAMPLE- COMPARISON/RELATIONAL OPERATORS

- Assume variable A holds 10 and variable B holds 20, then:

Operator	Description	Example
< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

JAVA LOGICAL OPERATORS

- **Logical operators** are used to determine the logic between variables or values.
- Assume variable A holds true and variable B holds false then:

Operator	Description	Example
Operator	Description	Example
&& (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
(logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A B) is true
! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true

TRUTH TABLE- LOGICAL AND (&&)

A	B	A&&B
True	True	True
True	False	False
False	True	False
False	False	False

TRUTH TABLE- LOGICAL OR (||)

A	B	A B
True	True	True
True	False	True
False	True	True
False	False	False

TRUTH TABLE- LOGICAL NOT (!)

A	!A
True	False
False	True

EXAMPLE-JAVA LOGICAL OPERATORS

- Assume a=true and b=false

Operator	Description	Example
&&	Logical AND	(a && b) is false
	Logical OR	(a b) is true
!	Logical NOT	(!a) is false

JAVA BITWISE OPERATORS

- **Bitwise operators** are used to perform binary logic with the bits of an integer or long integer.
 - **Bitwise operators** are used to perform manipulation of individual bits of a number.
 - They can be used with any of the integral types (char, short, int, etc.).
-

JAVA BITWISE OPERATORS

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
<<	left shift
>>	right shift

JAVA BITWISE OPERATORS

- **Bitwise OR (|)** –

This operator is binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e, if either of the bits is 1, it gives 1, else it gives 0.

For example:

```
a = 5 = 0101 (In Binary)
```

```
b = 7 = 0111 (In Binary)
```

```
Bitwise OR Operation of 5 and 7
```

```
  0101
```

```
| 0111
```

```
0111 = 7 (In decimal)
```

BITWISE OR (|)

BITWISE OPERATORS: & | ^ ~

Bitwise OR (|):

A	B	A B
1	1	1
1	0	1
0	1	1
0	0	0

Logical OR (||):

A	B	A B
T	T	T
T	F	T
F	T	T
F	F	F

System.out.println(38 | 23) == 55

	32	16	8	4	2	1
38 ₁₀	1	0	0	1	1	0
23 ₁₀	0	1	0	1	1	1
55	1	1	0	1	1	1

JAVA BITWISE OPERATORS

- **Bitwise AND (&) –**

This operator is binary operator, denoted by '&'. It returns bit by bit AND of input values, i.e, if both bits are 1, it gives 1, else it gives 0.

- For example:

```
a = 5 = 0101 (In Binary)
```

```
b = 7 = 0111 (In Binary)
```

```
Bitwise AND Operation of 5 and 7
```

```
  0101
```

```
& 0111
```

```
  0101 = 5 (In decimal)
```

BITWISE OPERATORS: & | ^ ~

Bitwise AND (&):

A	B	A&B
1	1	1
1	0	0
0	1	0
0	0	0

Logical AND (&&):

A	B	A&&B
T	T	T
T	F	F
F	T	F
F	F	F

```
System.out.println(38 & 23) == 6
```

	32	16	8	4	2	1
38 ₁₀	1	0	0	1	1	0
23 ₁₀	0	1	0	1	1	1
6	0	0	0	1	1	0

BITWISE
AND (&)

JAVA BITWISE OPERATORS

- **Bitwise XOR (^)** –

This operator is binary operator, denoted by '^'. It returns bit by bit XOR of input values, i.e, if corresponding bits are different, it gives 1, else it gives 0.

For example,

a = 5 = 0101 (In Binary)

b = 7 = 0111 (In Binary)

Bitwise XOR Operation of 5 and 7

0101

^ 0111

0010 = 2 (In decimal)

BITWISE XOR

BITWISE OPERATORS: & | ^ ~

Bitwise XOR (^):

A	B	A^B
1	1	0
1	0	1
0	1	1
0	0	0

Logical XOR (^):

A	B	A^B
T	T	F
T	F	T
F	T	T
F	F	F

```
System.out.println(38 ^ 23) == 49
```

	32	16	8	4	2	1
38 ₁₀	1	0	0	1	1	0
23 ₁₀	0	1	0	1	1	1
49	1	1	0	0	0	1

JAVA BITWISE OPERATORS

- **Bitwise Complement (~)** – This operator is unary operator, denoted by '~'. It returns the one's complement representation of the input value, i.e, with all bits inversed, means it makes every 0 to 1, and every 1 to 0. For example,

```
a = 5 = 0101 (In Binary)
```

```
Bitwise Compliment Operation of 5
```

```
~ 0101
```

```
1010 = 10 (In decimal)
```

BITWISE NOT ($\sim$)

Bitwise NOT ($\sim$):

A	$\sim A$
1	0
0	1

Logical NOT (!):

A	$\sim A$
T	F
F	T

JAVA BITWISE OPERATORS

```
public class operators {
    public static void main(String[] args)
    {
        //Initial values
        int a = 5;
        int b = 7;

        // bitwise and
        // 0101 & 0111=0101 = 5
        System.out.println("a&b = " + (a & b));

        // bitwise or
        // 0101 | 0111=0111 = 7
        System.out.println("a|b = " + (a | b));

        // bitwise xor
        // 0101 ^ 0111=0010 = 2
        System.out.println("a^b = " + (a ^ b));

        // bitwise and
        // ~0101=1010
        // will give 2's complement of 1010 = -6
        System.out.println("~a = " + ~a);

        // can also be combined with
        // assignment operator to provide shorthand
        // assignment
        // a=a&b
        a &= b;
        System.out.println("a= " + a);
    }
}
```



PROGRAM OUTPUT:

```
a&b = 5
```

```
a|b = 7
```

```
a^b = 2
```

```
~a = -6
```

```
a= 5
```

JAVA BITWISE OPERATORS

- **Signed Right shift operator (>>) –**

Shifts the bits of the number to the right and fills 0 on voids left as a result. The leftmost bit depends on the sign of initial number. Similar effect as of dividing the number with some power of two.

JAVA BITWISE OPERATORS

- **Left shift operator (<<)** –

Shifts the bits of the number to the left and fills 0 on voids left as a result.

Similar effect as of multiplying the number with some power of two.

JAVA BITWISE OPERATORS

Now lets see truth table for bitwise &, | and ^

a	b	a & b	a b	a ^ b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

The bitwise shift operators shifts the bit value. The left operand specifies the value to be shifted and the right operand specifies the number of positions that the bits in the value are to be shifted. Both operands have the same precedence. **Example**

a = 0001000

b= 2

a << b= 0100000

a >> b= 0000010

OPERATOR PRECEDENCE

- All operators inside the parenthesis are evaluated first.
 - $*$, $/$ and $\%$ are at the same level of precedence and are evaluated next.
 - $+$ and $-$ have the same level and are evaluated last.
 - When operators are at the same level, we use left to right associativity.
 - $3*7-6+2/4+6$ means $(((3*7) - 6) + ((2*5)/4)) + 6$.
-

OPERATOR PRECEDENCE

- Association – when selecting between two operators of the same priority they operate according to their associativity which is:
- Right to left – unary operators and assignment operators are right to left
- Left to right – all other operators

Precedence	Operators
Highest	()
Next	* , / , %
Lowest	+ , -

EXPRESSIONS

- These are the combinations of operators and operands.
 - *If all operands are integers this is called an integer expression* and it yields an *integer result*.
 - For example $2+3*5$.
 - And *if all operands are floating point this is called floating-point expression* and the *result is floating point numbers*.
 - Mixed expressions has operands of different data types. The result would be of the highest size data type.
 - For example, $2 + 3.5$.
-